NEA - Group Creator for School Projects

Amelie Machin

Contents

1 Analysis **2**
- 1.1 Problem statement . . . 2
- 1.2 Research . . . 2
  - 1.2.1 Sources of inspiration . . . 2
  - 1.2.2 Algorithm . . . 3
  - 1.2.3 Inputting data . . . 4
  - 1.2.4 Accounts . . . 4
  - 1.2.5 Internal data storage . . . 4
  - 1.2.6 External data storage . . . 5
  - 1.2.7 GUI library . . . 5
- 1.3 Dialogue . . . 5
- 1.4 Objectives . . . 7
- 1.5 Modelling . . . 12

2 Design **15**
- 2.1 GUI . . . 15
- 2.2 Databases . . . 22
- 2.3 Algorithm . . . 22
  - 2.3.1 Pseudocode . . . 22
  - 2.3.2 Fitness algorithm . . . 24
  - 2.3.3 Diagrams . . . 24

3 Technical solution **26**
- 3.1 main.py . . . 26
- 3.2 algoClasses.py . . . 36
- 3.3 geneticAlgo.py . . . 37

4 Testing **38**

5 Evaluation **42**

1 Analysis

1.1 Problem statement

To form the perfect grouping for a group project within a school, many factors need to be taken into consideration to prevent students from feeling incapable of completing the project. These factors include, but are not limited to, skills of the students, skills required by the project, and number of students in the group.

This project aims to create a tool to aid teachers in ensuring the students are separated into groups where the students are best able to get along and combine their individual skills to achieve their project's aims. The user can input the data that they want to be taken into consideration by the program and it will output a set of groups. This is in the form of a table on a GUI - to present the results in a readable way - and an option to export the data.

This program will not find, assume, or generate information about the students or project, it will only work with the data inputted to assign a group of students to a project.

Teachers will benefit from this tool as they don't have to spend as much time or effort creating groups for the projects they set for their students, so they can spend their time and effort on helping and teaching the students instead. Also, when the students are working on their projects, they are more likely to be focused on their work, so require less oversight from the teacher.

Students will also benefit from this tool as they would be able to apply their skills to a project in a group of people they can enjoy time with, rather than being stuck on a project they have no interest in with people they do not get along with. This means they are more likely to enjoy their time at school, be able to get more work done, and therefore be able to learn more, as opposed to being unmotivated and distracted in a poorly made group.

1.2 Research

1.2.1 Sources of inspiration

1. Random group generator from ClassTools.net [Fig. 1]
   https://www.classtools.net/random-group-generator/
   - This tool is used frequently by my teachers to create random groups for group activities.
   - The GUI is very simple to use as there is little data that needs to be entered, so can all fit down the left of the screen.
   - The output is very easy to read as each group is visually separated using both coloured boxes and numbering. [Obj. 3b]
   - However, this tool is too simple because it lacks any way to group with conditions, which is part of my scope, as they are purely randomised. This means that teams could quit often be unfairly balanced in terms of skill, or group students together that are known

to not work well together. The teacher's only option to solve this is to re-roll until the output is suitable enough for the task manually. This gave me the idea for the algorithm to be the simulation of this re-rolling and selection of optimal outputs.

2. keamk [Fig. 2] [Fig. 3]
   https://www.keamk.com/random-team-generator

   - This tool creates groups that all have a balanced level of skill.
   - I like how the skill level is inputted as a rating between 1-5 as this is intuitive for the user.
   - I also like the extra details that are outputted, such as how many are in each group next to the group name and the skill levels next to each individual. [Obj. 3a]
   - Furthermore, the option to go back to make small changes to the data inputted and get new groupings was very quick and useful. [Obj. 1g]
   - However, this tool can only create balanced groups, not groups with different objectives as is in my scope, and there is no way to assign a certain number of participants to each group, as the program puts the same number (or one away) of participants in each group. [Obj. 2c-d]

These tools influenced me to reduce the complexity of my GUI, and gave me ideas such as to keep groups visually separate using colour as well as tables, adding the option to go back and edit the inputted data later, and some summary statistics to include with the outputs.

1.2.2 Algorithm

I have researched into representing students and projects into bipartite graphs, and then using matching algorithms such as the alternate path algorithm to join students to projects [Fig. 4]. However, this will only match one student to one project, not groups so is not suitable. I then researched applying the algorithm repeatedly to graphs with randomised students, but this means that some students will never be joined into a group purely based on chance and that the groups formed are such that each individual is best for the project, not that the overall group is best suited for the project, so teamwork will not as good as it could be. I then researched using a grouping algorithm, such as https://help.feedbackfruits.com/hc/en-us/articles/23527084211346-Group-Formation-Algorithm-Explainer to create similar groups then using bipartite matching algorithms to match them. However this only creates groups with similar skill sets, not skill sets that vary based on the project.

I decided on a genetic algorithm where each individual is made up of a possible solution and the fitness score is an average between how suited each group is to its assigned project [Fig. 5]. The individuals with higher fitness scores are discarded and the lower ones are randomly selected to crossover to form new

individuals. Some of these individuals formed may contain error such as the same student multiple times, or a missing student, so that must be corrected before continuing. Then, random mutations are applied to some individuals, which may involve swapping of students between groups, moving some students to an optional project, or moving some students to group that isn't full. New fitness scores are calculated and the process repeats a predetermined number of times [Fig. 9]. I chose this because genetic algorithms are very adaptable to varying conditions and constraints, such as the ones that the user chooses in this program. This will allow the program to test different combinations of projects and students, test different group sizes for a project, and swap in and out optional projects to find better and better solutions until it reaches a high-quality solution that fits all the user's needs. This fits the project scope far better than all previously researched algorithms.

1.2.3 Inputting data

For the inputting of data, I have decided that it can be inputted through txt and csv files [Obj. 1a], manually on the GUI [Obj. 1b], or any combination of those methods [Obj. 1c]. This is because the data for different individuals and different projects could be stored in different ways within the same system, and manual tweaks may be needed by the user to ensure all data is as is needed by the application. I will also include the option to import the data from an Excel spreadsheet at the request of my third party [Obj. 1b].

1.2.4 Accounts

I have decided against implementing an account system because I don't think it will provide any value to the program. Accounts are typically used for ensuring data can only be accessed by the users with the correct permissions. As this program will be designed to be run locally, all the data will be stored and only accessible on the machine the program was run on, so the OS can handle user security.

Another reason for accounts is allow users to access their data that has been stored by a third party, especially within a cloud to allow for data to be quickly transferred between devices. I will implement a manual solution to this by allowing users to export and import all the data the program stores and the user can handle that data transfer themselves.

1.2.5 Internal data storage

I chose to store the data about the students and projects in their own classes, that are aggregated within an 'individual' class. The student and project classes contain their names, skills and the project will have the recommended group size. The individual will contain a project object, student objects, and current fitness score as well as methods for calculating fitness scores, crossover with other individuals, and mutations. [Fig. 7]

This keeps the attributes and methods stored together in the program to make the code easier to read and understand by a programmer.

1.2.6 External data storage

[Obj. 4] I have researched into whether information about projects and students should be stored somewhere, and if so, how. I have concluded that they should be stored in relational databases that are accessible and modifiable by entering the details into the graphical interface, which are passed as parameters into SQL statements. These can then be loaded as a list of checkboxes that can be selected and deselected by the user for that run. [Obj. 1c]

I chose to store the data because being able to make minor changes to the data and then re-running the program quickly is a very valuable feature that could save users' time and energy as they would not have to go through the entire process of entering every piece of data just for small changes to some of the data. This feature is present in keamk's grouping tool, and I found it very useful in my testing.

I chose databases to store the data because I have more experience working with relational databases and SQL then any other form of data storage, such as in files. The data is structured so databases would allow for efficient access and modification of specific entries.

1.2.7 GUI library

Between the two main python GUI libraries, pygame and Tkinter, I chose Tkinter because of the larger library of classes of pre-made, customisable widgets that can be used more easily to produce a simple, easy-to-use tool, such as a dialog that allows users to graphically navigate through their files to find their data file [Fig.6].

On the other hand, with my prior experience with pygame, I have found it to be more suited towards building games and graphically-complex tools, which is not what I am aiming for in this project. I am aiming to have a GUI that is quick and simple to create and edit, and has user-friendly features for inputting, editing, and reading data. I also have more experience with Tkinter, which will reduce the amount of time I spend on the GUI and allow me to focus on developing and improving the grouping algorithm and user experience.

1.3 Dialogue

My third party is Dr Ed Horncastle, the lead teacher for the Exeter Mathematics Certificate (EMC) at my school, which is where groups of 3 or 4 students work together on research projects set by University academics or companies. He is suitable because he has to make these groups based on the skills of the students and match them to projects every year, so he has a thorough understanding of how this tool could work and would benefit from using this tool. I interviewed him and asked him the following questions:

1. What is the current process for creating EMC groups?
    - All the metrics to consider are gathered into a spreadsheet and then groups are formed by hand.
    - Some projects don't end up getting groups assigned, especially those that have been set by a company that already has another project with an assigned group.
2. What are these metrics?
    - Students' own choice
    - There must be a mix of genders
    - The projects must be relevant to the A-level studies of that student. Someone who does not take computer science shouldn't be given a computer science based project
    - Those that work hard and those that laze off more easily should be mixed together. If this is their first project, this is inferred from GCSE results, otherwise it is based off the results of previous projects.
    - Special educational needs
    - Timetabling restrictions
    - Students do not get put into groups with others they have worked on projects with before
3. How are the student choices measured?
    - For some of the projects, the students give their top 3 choices of projects. Starting with the least popular project, the groups are formed by trying to minimise the sum of the positions the student's ranked the project in.
4. How is the data about the students and projects currently stored?
    - Excel spreadsheet
5. How would you prefer to input the data?
    - Student choices from a Microsoft Form and all other data from a Excel spreadsheet
6. How much data and in what format do you think the data should be outputted?
    - In tables with the project name, student names and summaries of information about each group such as gender ratio, how many take computer science or physics, and how many have special education needs.
    - As a simplified register in alphabetical order

7. How useful would storing project/student details locally, that can be loaded when needed, be for you?

    - Very

8. Are there any other features that would be useful for this tool to include?

    - Some extra features that could be added is timetabling for what order the projects are presented in, and itinerary for group trips out of school.

After this, I decided to include Excel spreadsheets as an input method [Obj. 1b], include options for projects to not have groups assigned despite being a part of the algorithm [Obj. 1f], take genders into account [Obj. 2e], have summary statistics presented alongside the groups [Obj. 3a], have the option to view the groups without any extra information [Obj. 3b], store the data locally for re-use at another time [Obj. 4].

I decided not to include student choice, student personality, special education needs, timetabling restrictions, or preventing grouping the same students together across multiple as adding too many restrictions to the algorithm will make it take it longer to run to create a set of groups that best matches all of these restrictions. These are the restrictions that were lower on Dr Horncastle's priorities. I will also not include timetabling or scheduling as these deviate quite far from my project's scope.

1.4 Objectives

1. Data can be entered into the program from external sources
    (a) Data can be entered manually into the GUI
    (b) Data can be entered through txt, csv, and Excel spreadsheet files
    (c) Data can be loaded from stores from previous uses of the program
    (d) Data can be entered through a combination of other methods
    (e) Student data must contain a name, gender, and skills
        i. Name must only contain alphanumeric characters and certain whitelisted characters
    (f) Project data must contain a name, maximum group size, minimum group size, whether or not the project must have a group or not, and skill requirements
        i. Name must only contain alphanumeric characters and certain whitelisted characters
        ii. Maximum group size must be greater than 0
        iii. Minimum group size must be greater than or equal to 0
    (g) Data can be added, edited, and removed at any time during the program's runtime

(h) Data must be entered before the algorithm is run

2. Students must be grouped together and to a project

   (a) All students must be put into a group

   (b) All students must have at least one skill that will allow them to contribute to the project, when possible

   (c) Projects must not have more students assigned to it than the maximum limit inputted by the user

   (d) Projects must not have less students assigned to it than the minimum limit inputted by the user

   (e) Groups must have a mix of genders

3. The resulting groups can be viewed

   (a) This can be through coloured tables displayed on the GUI that contain the project name, student names, number of students, gender ratio, and skill range

   (b) This can be through a register that displays projects in alphabetical order, grouped with its students in alphabetical order, all coloured to separate from each other, and no other information.

   (c) Both above options must automatically be saved as ... files in the local file directory.

4. All data about projects and students must be stored

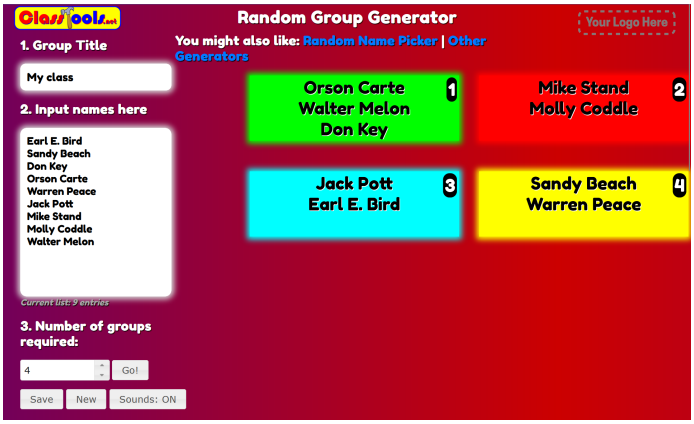


Figure 1: ClassTools's random group generator tool with inputs and outputs visible on the same page

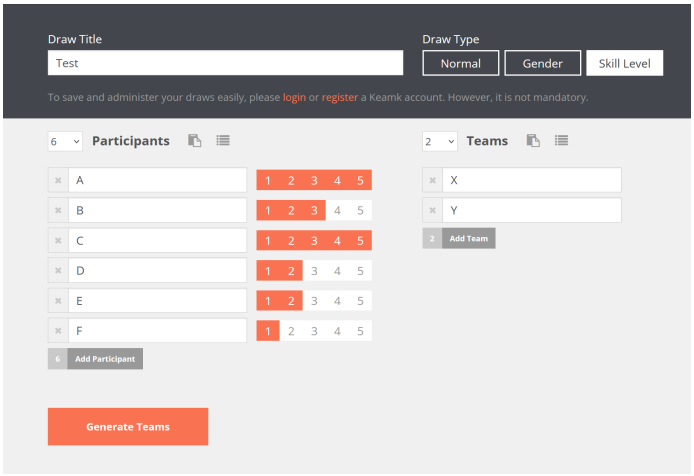


Figure 2: Keamk's menu for inputting data



Figure 3: Keamk's outputs

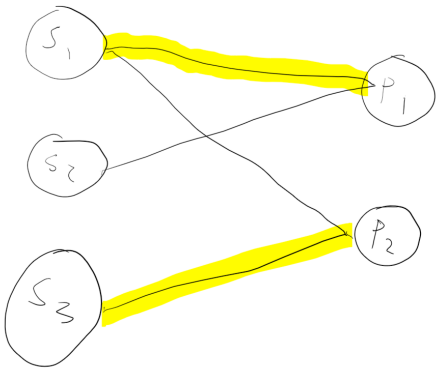


Figure 4: A bipartite graph with 3 student nodes on the left, 2 project nodes and the right, and some edges between them. A matching algorithm has been applied, with the matching highlighted in yellow. This has not met the scope because a student has not been matched.

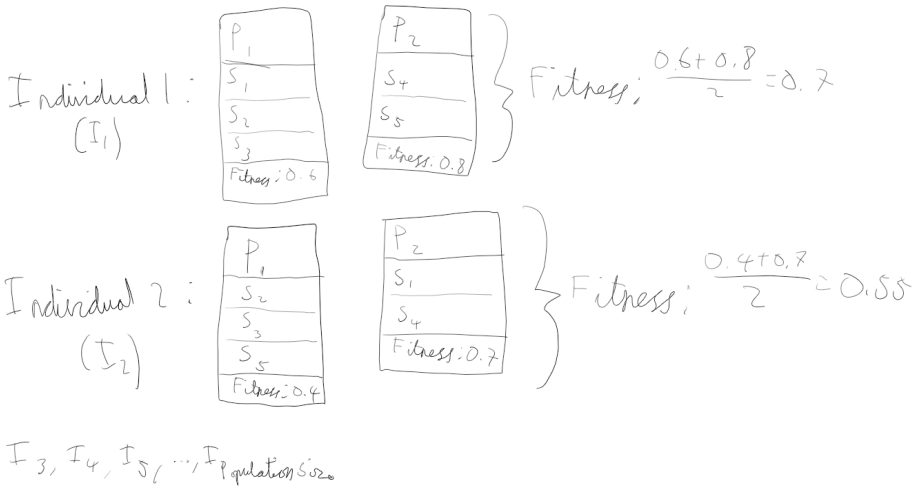


Figure 5: A sketch showing an example of a population, representing a set of possible solutions (individuals). Each individual represents a possible set of groups, and has a fitness score that represents how well every project's group matches the project's and user's requirements. This is calculated by averaging all the fitness scores of the groups it contains.

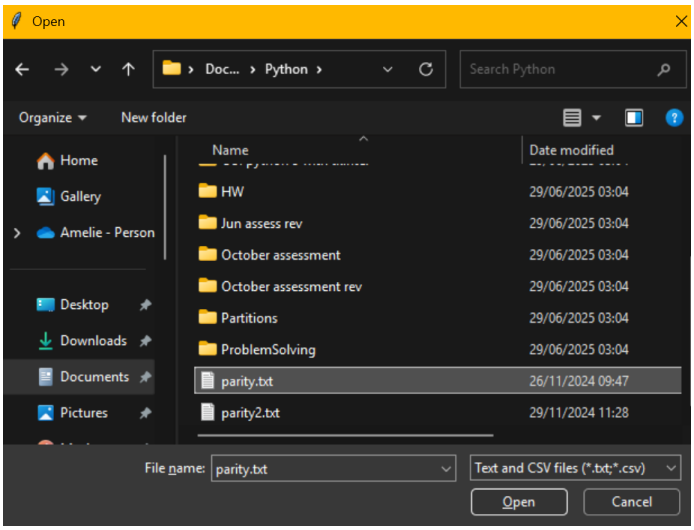


Figure 6: Tkinter's file dialog, configured to only show .txt and .csv files

1.5 Modelling

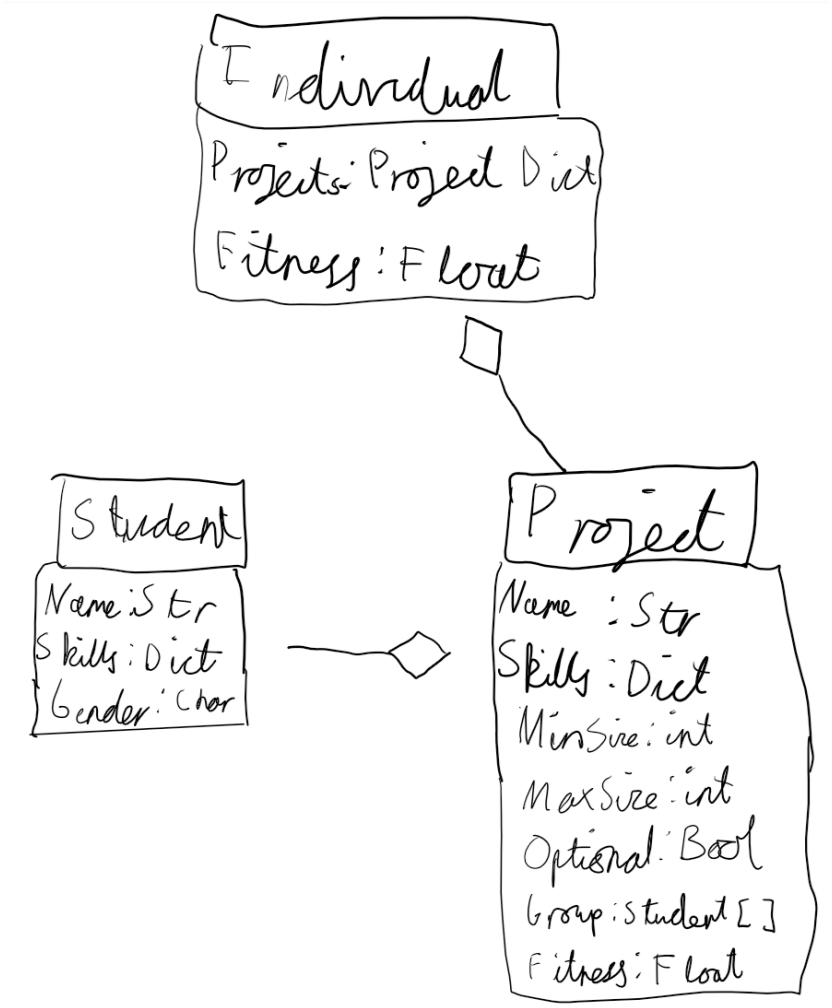


Figure 7: Class diagram showing relationships between the classes and their attributes used by the genetic algorithm.

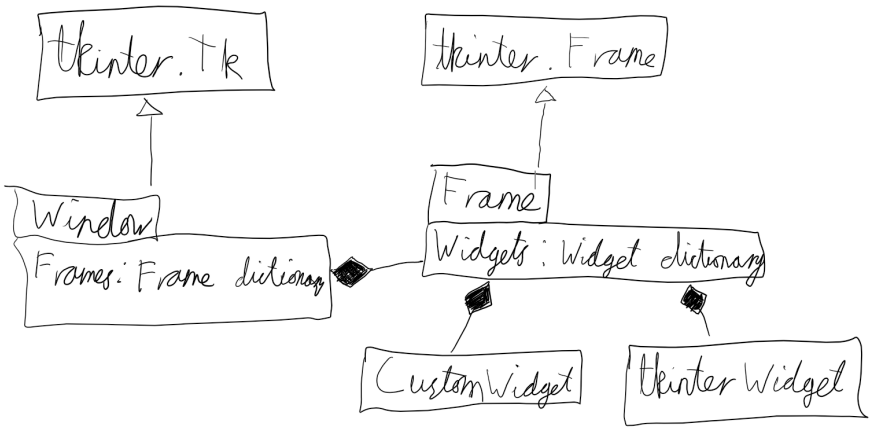


Figure 8: Class diagram showing relationships between the classes that form the GUI

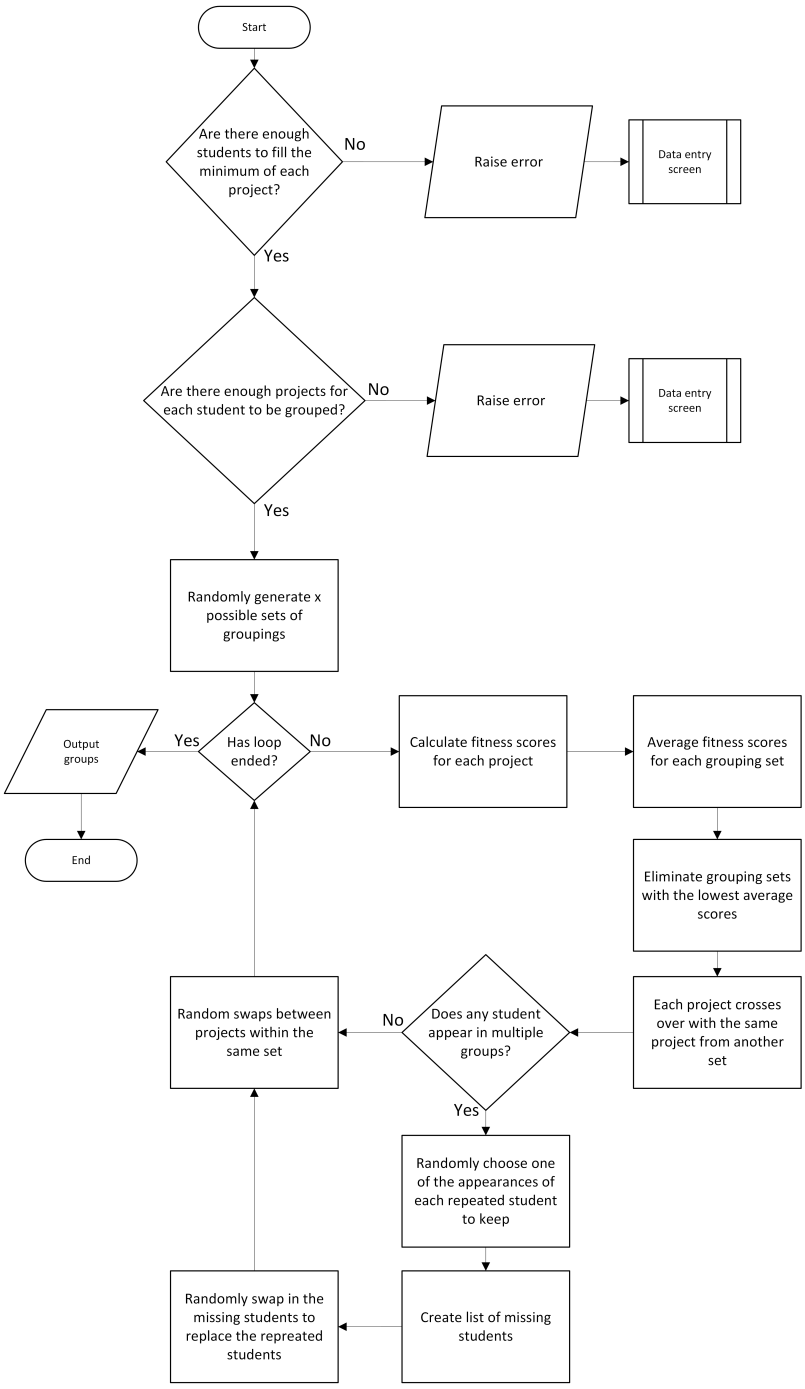


Figure 9: Flowchart showing the process for the genetic algorithm

2 Design

2.1 GUI

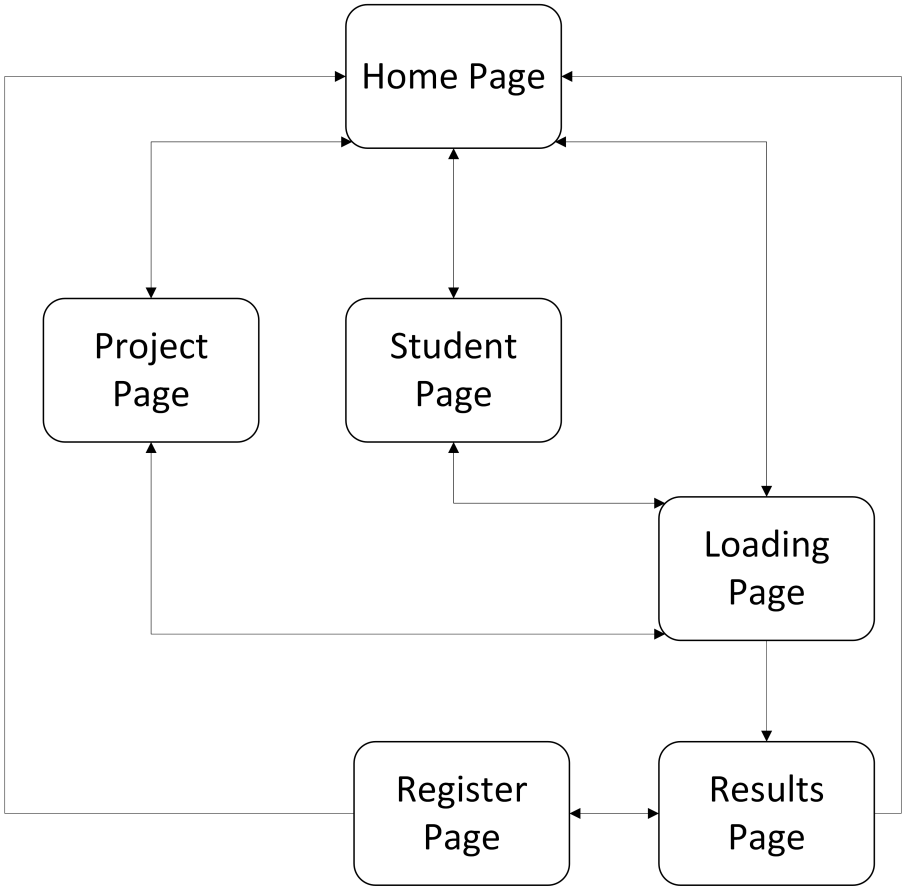


Figure 10: Page navigation diagram showing which pages can navigate to each other. Note: The student and project pages can only navigate to the previous page.

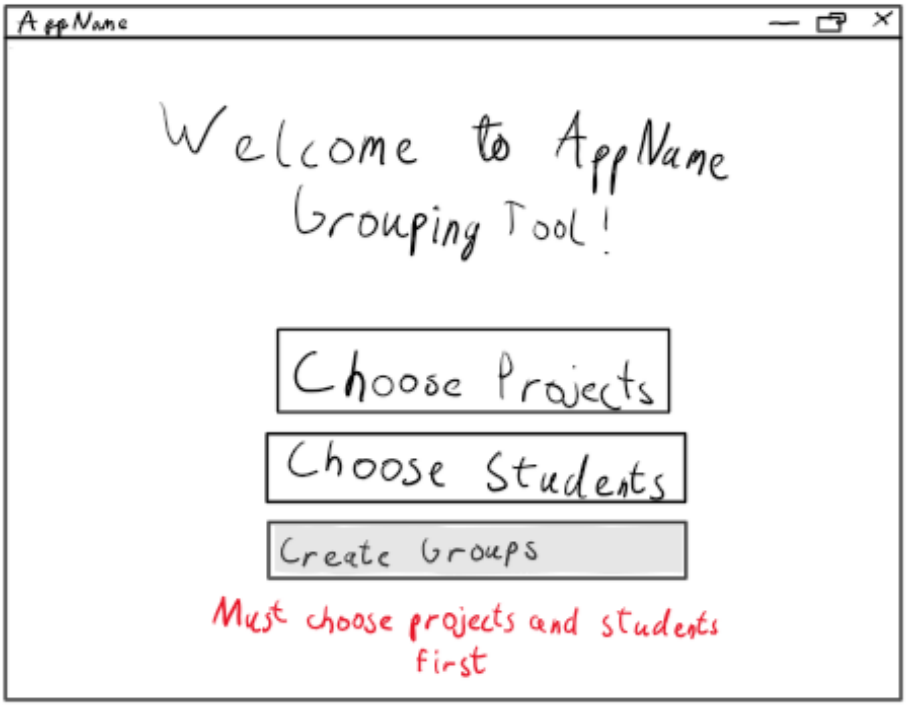


Figure 11: Sketch of the homepage layout that is the first page seen upon opening the app. The "Choose Projects" and "Choose Students" lead to the projects and students pages, respectively. The "Create Groups" button leads to the loading page and runs the grouping algorithm with the students and projects the user has selected. It is greyed out and not clickable unless at least one project and student has been selected [Obj. 1h].

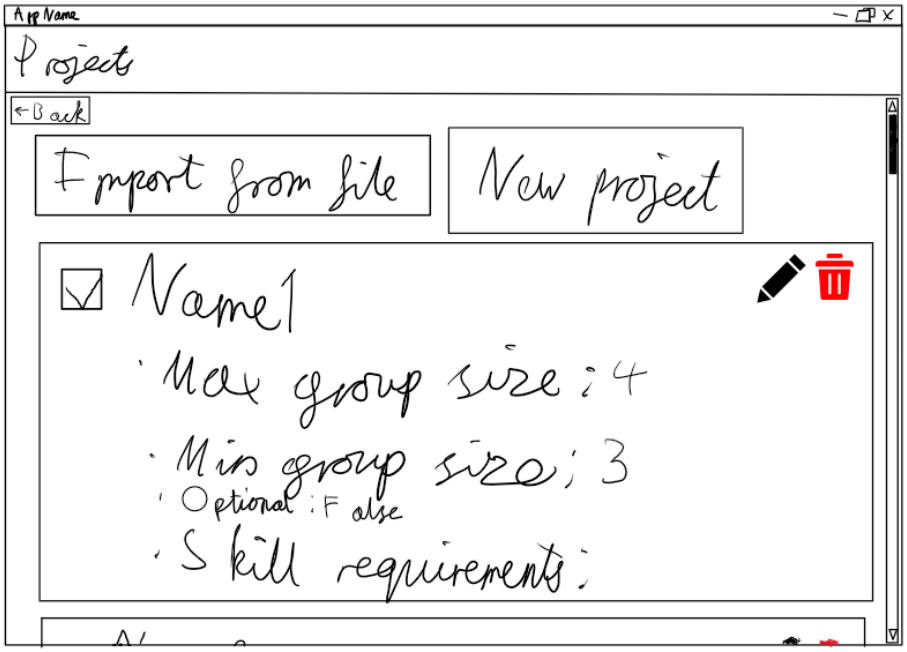


Figure 12: Sketch of the project page layout. The projects in the scrollable list are all those that are currently stored in the database [Obj. 1c]. There are options to import project data from supported files [Obj. 1b] and to manually enter project details at the top [Obj. 1a], which then get added to the database [Obj. 4] and the visual list [Obj. 1d]. The project details can also be edited or deleted from the database by pressing the buttons [Obj. 1g], which run parametrised SQL. Each project in the list features a checkbox, allowing users to select which of the projects in the database will be used in the grouping algorithm. Projects not selected will not have groups matched to them. This page also features a back button to go back to the home page.

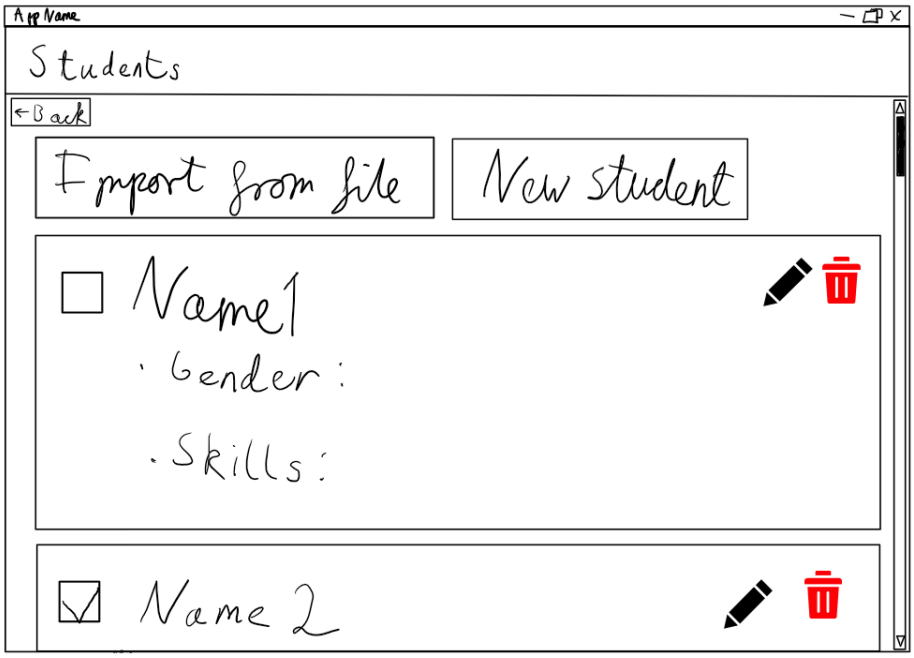


Figure 13: Sketch of the student page layout. The students in the scrollable list are all those that are currently stored in the database [Obj. 1c]. There are options to import student data from supported files [Obj. 1b] and to manually enter student details at the top [Obj. 1a], which then get added to the database [Obj. 4] and the visual list [Obj. 1d]. The student details can also be edited or deleted from the database by pressing the buttons [Obj. 1g], which run parametrised SQL. Each student in the list features a checkbox, allowing users to select which of the students in the database will be used in the grouping algorithm. Students not selected will not be put into groups. This page also features a back button to go back to the home page.

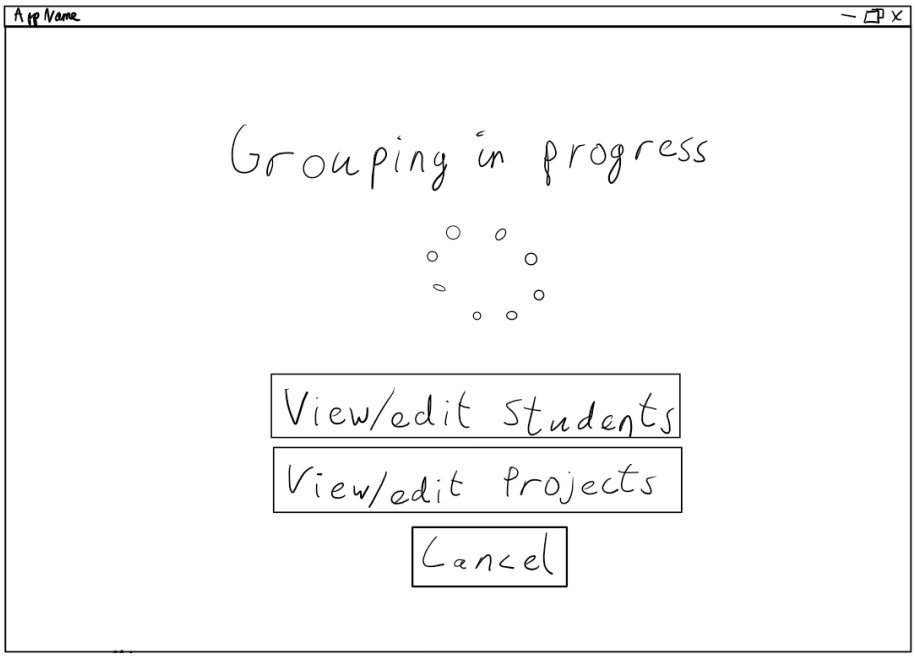


Figure 14: Sketch of the loading page layout. This page features a loading animation, and three buttons. The "View/edit Students" and "View/edit Projects" buttons lead to the Students and Projects pages, respectively [Obj. 1g]. The "Cancel" button ends the algorithm and leads to the home page. Once the algorithm has finished successfully, it moves onto the results page.

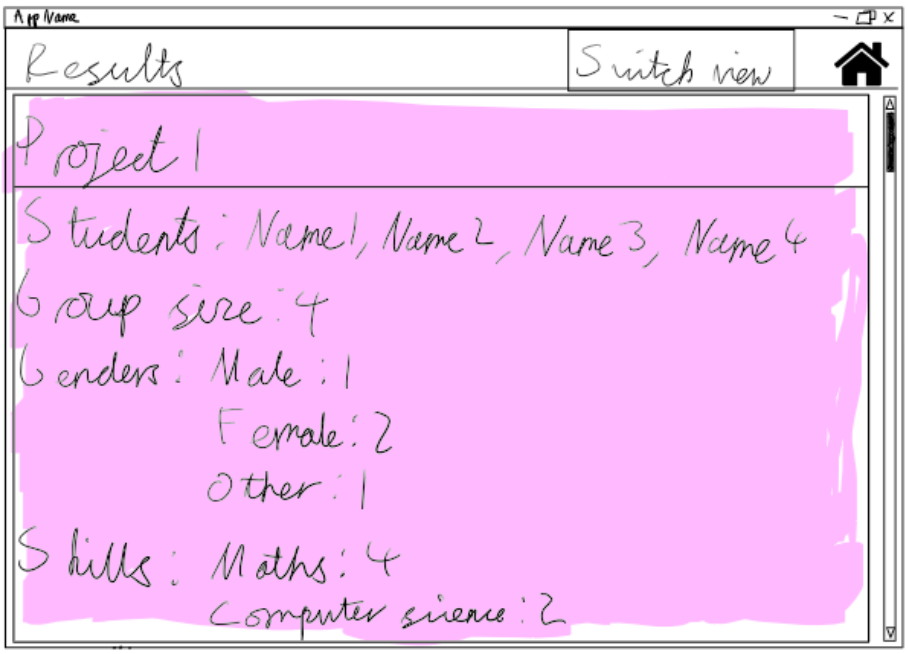


Figure 15: Sketch of the results page layout. It features a scrollable list of all the projects with the students grouped to it and summary statistics about the group, such as group size, gender ratio, and skills [Obj. 3a]. The "Switch view" leads to the register page and the home button leads to the home page.

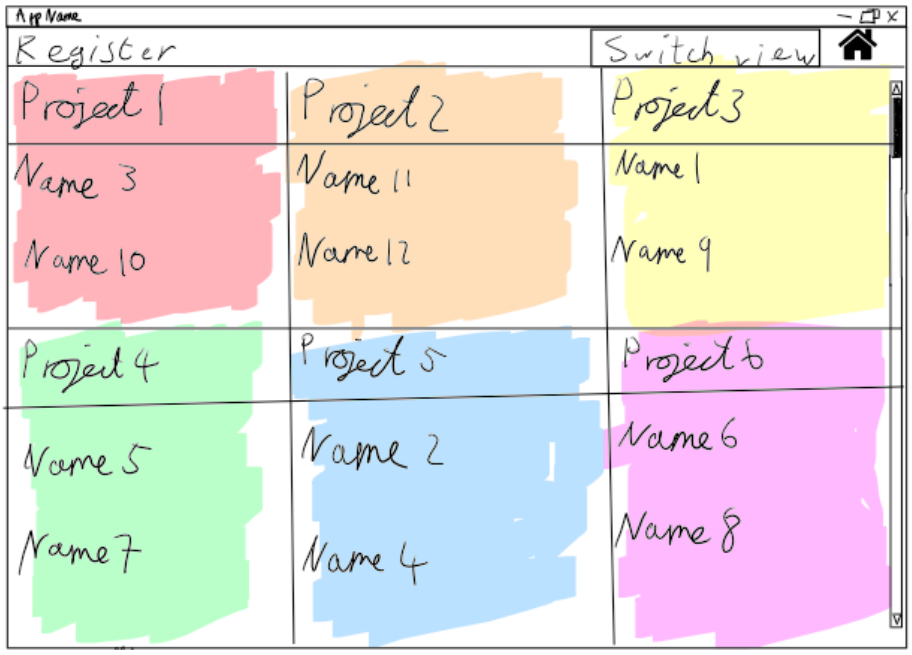


Figure 16: Sketch of the register page layout.

2.2 Databases

- Students(StudentID, Name, Gender)
- StudentSkills(StudentID, Skill)
- Projects(ProjectID, Name, MaxSize, MinSize, Optional)
- ProjectSkills(ProjectID, Skill)

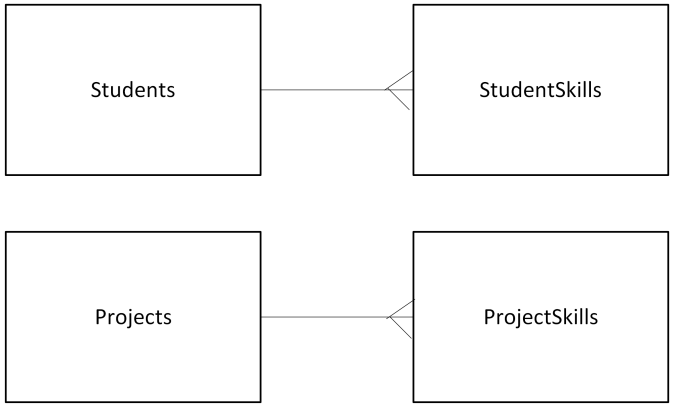


Figure 17: An entity relationship diagram showing the one-to-many relationships between the Students and Projects entities and the store of their skills to ensure all fields are atomic so the database is normalised.

2.3 Algorithm

2.3.1 Pseudocode

Listing 1: Pseudocode for the function that runs when on the loading page. It ensures it is possible to find groupings with the given requirements, then runs a genetic algorithm to find a good set of groupings.

```
1 TARGET_FITNESS = integer
2 MAX_ITERATIONS = integer
3 POPULATION_SIZE = integer
4
5
6 students = set of 'student' objects
7 projects = set of 'project' objects
8
9
```

```
10 #Ensure there aren't too many or too few students to meet
       ↪ requirements
11 minStudents = 0
12 maxStudents = 0
13
14 REPEAT for all projects:
15     maxStudents += project.maxSize
16     IF (not project.optional):
17         minStudents += project.minSize
18
19 #Check for too few students
20 IF (length of students < minStudents):
21     DisplayError("Error: Only {length of students} to
           ↪ fill a minimum of {minStudents} places.")
22     HomePage()
23
24 #Check for too many students
25 ELSE IF (length of students > maxStudents): #Checks for
       ↪ too many
26     DisplayError("{length of students} is too many to fit
           ↪ in {maxStudents} places.")
27     HomePage()
28
29
30 #Generate initial grouping sets and average fitness
       ↪ scores
31 population = list
32 REPEAT for i = 0 to POPULATION_SIZE - 1:
33     groups = GenInitialGroups(projects, students)
34     avgFit = (SUM fitness of each group)/(length of
           ↪ groups)
35     APPEND groups,avgFit to population
36
37 SORT population by avgFit
38
39 #Genetic algorithm
40 REPEAT for i = 1 to MAX_ITERATIONS:
41     #Ends early when one set of groupings is good enough
42     IF (highest avgFit > TARGET_FITNESS):
43         BREAK
44
45     #Eliminate the worst half of grouping sets
46     population = first (POPULATION_SIZE FLOORDIV 2) of
           ↪ population
47     new_pop = []
48
```

```
49      #Create next population
50      REPEAT for j = 1 to 90% of POPULATION_SIZE:
51          #Crossover random groupings
52          groups = Mate(random group from population,
                ↪ random group from population)
53
54          #Ensure all students appear once
55          valid = verifyGroupings(groups)
56          IF (not valid):
57              fixGroupings(groups)
58
59          #Random mutations to prevent convergence towards
                ↪ a local best solution
60          Mutate(groups)
61
62          #Find the average fitness score for each new
                ↪ grouping
63          avgFit = (SUM fitness of each group)/(length of
                ↪ groups)
64          APPEND groups,avgFit to new_pop
65
66      #Combine the crossover with 10% elitism
67      population = new_pop + top 10% of population
68      SORT population by avgFit
69
70
71  return group with highest avgFit
```

2.3.2 Fitness algorithm

Still working on it.

2.3.3 Diagrams

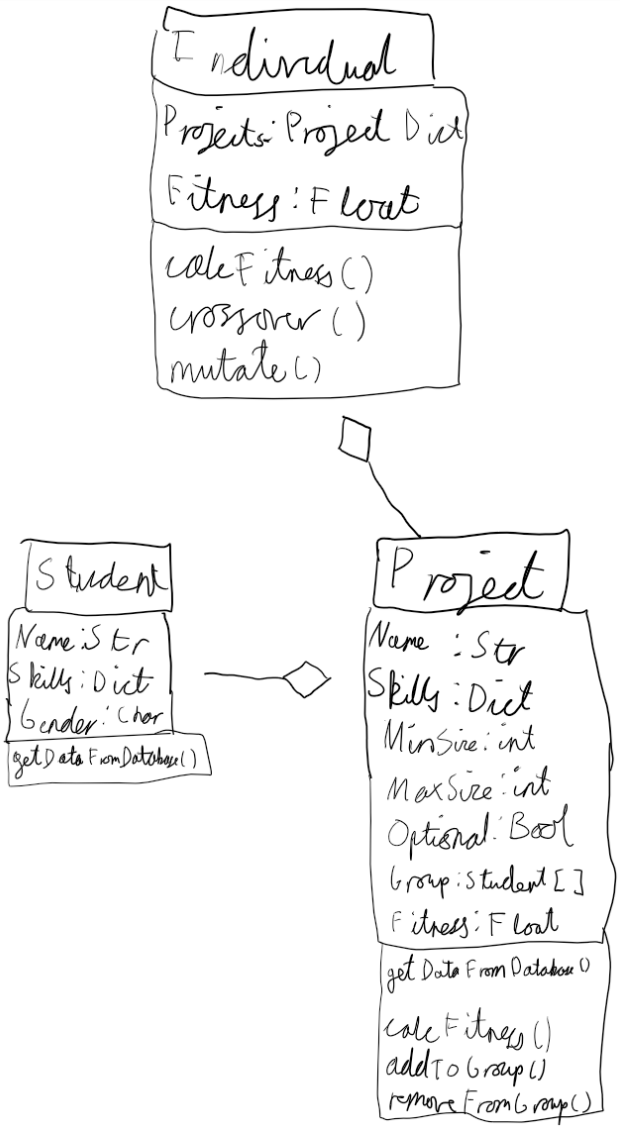


Figure 18: A class diagram showing the attributes and methods of the classes used by the genetic algorithm, and the relationships between each class.

3 Technical solution

Contents

3.1 main.py 26
3.2 algoClasses.py 36
3.3 geneticAlgo.py 37

main.py is the main file that is run by the user. It contains the GUI, and calls to the functions and classes in the other files when needed. All the pages outlined above [Fig. 10] have their own classes, whose objects are stored in the window class. The page classes' attributes are mostly widgets, both tkinter widgets and custom widgets (such as Header, ScrollableFrame, SelectionBox, and ProjectBox), or the attributes store the result of user interaction, such as the state of the checkboxes on the project page. The methods all code for the processing and outputting the result of human interaction with the GUI.

algoClasses.py contains the Individual, Student, and Project classes for use by the genetic algorithm. These follow the design of the above class diagram [Fig. 18].They are stored in a separate file for re-use across multiple files.

geneticAlgo.py contains the code for genetic algorithm that calculates the groups. it follows the design of the above pseudocode [Listing 1]

3.1 main.py

```
import tkinter as tk
from tkinter import ttk
from tkinter import filedialog
from tkinter import messagebox

#=====GLOBAL CONSTANTS=====

BG_COLOUR="#FFFFFF"
HEADER_COLOUR="#FFDB32"
LARGE_FONT=("Ariel",30)

#=====ROOT WINDOW=====
#Constants for styling included within

class Window(tk.Tk):
    def __init__(self):
        #Setup window
        tk.Tk.__init__(self)
        self.state('zoomed')
```

```
20          self.geometry(f"{self.winfo_screenwidth()-10}x{self.
    ↪ winfo_screenheight()}")
21          self.title('Student Grouping Tool')
22          self.config(background=BG_COLOUR)
23
24          #=====STYLES=====
25
26          HEADER_BUTTON_STYLE=ttk.Style()
27          HEADER_BUTTON_STYLE.configure("Header.TButton",background=
    ↪ HEADER_COLOUR)
28
29          #================
30          #Dictionary that will store all loaded pages
31          self.frames={}
32          #Show the home page first
33          self.show_frame(HomePage)
34
35      #Method for changing currently open page
36      def show_frame(self, f):
37          #If the page is already loaded, show it
38          if f in self.frames.keys():
39              frame = self.frames[f]
40          #Otherwise, load it then show it
41          else:
42              frame = f(self)
43              self.frames[f] = frame
44              frame.place(relx=0,rely=0,relwidth=1,relheight=1)
45          frame.tkraise()
46
47  #=====PAGES=====
48
49  class HomePage(tk.Frame):
50      def __init__(self,parent):
51          tk.Frame.__init__(self, parent)
52          self.label = ttk.Label(self, text="Welcome to AppName Grouping
    ↪ Tool!", font=LARGE_FONT)
53          self.label.pack()
54
55          self.projectButton = ttk.Button(self,text="Choose Projects",
    ↪ command=lambda:parent.show_frame(ProjectPage))
56          self.projectButton.pack()
57
58          self.studentButton = ttk.Button(self,text="Choose Students",
    ↪ command=lambda:parent.show_frame(StudentPage))
59          self.studentButton.pack()
60
61          self.runButton = ttk.Button(self,text="Create Groups",command=
    ↪ lambda:parent.show_frame(LoadingPage),state="disabled")
62          self.runButton.pack()
63
```

```
64          self.warning = ttk.Label(self,text="Must choose projects and
    ↪ students first")
65
66
67      #This method runs once everytime the page is opened
68      def tkraise(self):
69          tk.Frame.tkraise(self)
70
71          #If at least one student and one project has been selected, allow
    ↪  the run button to be clickable and remove the warning message
72          if ProjectPage in self.master.frames and StudentPage in self.
    ↪ master.frames and self.master.frames[ProjectPage].
    ↪ check_for_selection() and self.master.frames[StudentPage].
    ↪ check_for_selection():
73              self.runButton["state"]="enabled"
74              self.warning.pack_forget()
75
76          #Otherwise, disable the run button and display the warning
    ↪ message
77          else:
78              self.runButton["state"]="disabled"
79              self.warning.pack()
80
81
82  class ProjectPage(tk.Frame):
83      def __init__(self,parent):
84          tk.Frame.__init__(self,parent)
85
86          self.header=Header(self,"Projects",HEADER_COLOUR)
87          self.header.pack(fill="x")
88
89          self.backButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage)) #Change to
    ↪ check where it should go back to
90          self.backButton.pack(side="right")
91
92          self.importButton=ttk.Button(self,text="Import from file",command
    ↪ =self.import_projects)
93          self.importButton.pack()
94
95          self.addButton=ttk.Button(self,text="New project",command=self.
    ↪ new_project)
96          self.addButton.pack()
97
98          #self.separator=ttk.Separator(self,orient="horizontal")
99          #self.separator.pack(fill="x",expand=True)
100
101         self.boxesFrame=ScrollableFrame(self)
102         self.boxesFrame.canvas.pack(fill="both",expand=True)
103         self.update()
```

```
104
105         self.projectBoxes={}
106         for i in range(10):
107             name="Name"+str(i)
108             self.projectBoxes[name]=projectBox(self.boxesFrame,self,name,
    ↪ 0,0,True,["g"])
109             self.projectBoxes[name].pack(fill="x",padx=10,pady=5)
110 
111     #Method override as a work-around for a scrolling bug
112     def update(self):
113         self.master.state('normal')
114         tk.Frame.update(self)
115         self.master.state('zoomed')
116 
117     #Iterates through every project, and returns true if it finds a
    ↪ selected project. Returns false when no project is selected.
118     def check_for_selection(self):
119         for box in self.projectBoxes:
120             if self.projectBoxes[box].check_selection():
121                 return True
122         return False
123 
124     def import_projects(self):
125         #Show an "Open" dialog box and return an open file object in read
    ↪  mode
126         file = filedialog.askopenfile(mode='r',filetypes=[("Supported
    ↪ file types",".txt .csv")]) #Add excel
127         #Validation to ensure the user selected a file
128         if file == None:
129             return
130 
131         #<<<-------------------------------------------
132 
133         file.close()
134         return
135 
136     def new_project(self):
137 
138         popupWin=entryPopupWin(self,"New project")
139         name=tk.StringVar()
140         maxSize=tk.StringVar()
141         minSize=tk.StringVar()
142         optional=tk.BooleanVar()
143         skills=tk.StringVar()
144 
145         popupWin.add_string_entry("Enter name:",name)
146 
147         popupWin.add_int_entry("Enter the maximum size allowed for this
    ↪ project's group:",maxSize)
148 
```

```
149         popupWin.add_int_entry("Enter the minimum size allowed for this
    ↪ project's group:",minSize)
150 
151         popupWin.add_checkbox("Is this project optional?",optional)
152 
153         popupWin.add_string_entry("Enter skills, each one separated by a
    ↪ comma:",skills)
154 
155         popupWin.add_save_cancel(popupWin,name,maxSize,minSize,optional,
    ↪ skills)
156 
157     def save_entry(self,window,*variables):
158         name=variables[0].get()
159         maxSize=variables[1].get()
160         minSize=variables[2].get()
161         optional=variables[3].get()
162         skills=variables[4].get()
163 
164         #Validation to ensure all required entries are filled
165         if name != "" and maxSize != "" and minSize != "" and skills != "
    ↪ ":
166 
167             maxSize = int(maxSize)
168             minSize = int(minSize)
169 
170             #Separate skills into a list and remove whitespace
171             skills=skills.split(",")
172             for i in range(len(skills)):
173                 skills[i]=skills[i].strip()
174 
175             #Display new project on the GUI
176             self.projectBoxes[name]=projectBox(self.boxesFrame,self,name,
    ↪ maxSize,minSize,optional,skills)
177             self.projectBoxes[name].pack(fill="x",padx=10,pady=5)
178 
179             #Destroy the popup window
180             window.destroy()
181 
182             #Work-around for a scrollbar bug
183             self.update()
184 
185     def editBox(self,boxName):
186         return
187 
188     def deleteBox(self,boxName):
189         self.projectBoxes[boxName].pack_forget()
190         del self.projectBoxes[boxName]
191         self.update()
192 
193 
```

```
194
195
196 class StudentPage(tk.Frame):
197     def __init__(self,parent):
198         tk.Frame.__init__(self, parent)
199
200         self.header=Header(self,"Students",HEADER_COLOUR)
201         self.header.pack(fill="x")
202
203         self.backButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage)) #Change to
    ↪ check where it should go back to
204         self.backButton.pack(side="right")
205
206         self.importButton=ttk.Button(self,text="Import from file",command
    ↪ =self.import_students)
207         self.importButton.pack()
208
209         self.addButton=ttk.Button(self,text="New student",command=self.
    ↪ new_student)
210         self.addButton.pack()
211
212         self.boxesFrame=ScrollableFrame(self)
213         self.boxesFrame.canvas.pack(fill="both",expand=True)
214         self.update()
215
216         self.studentBoxes={}
217         for i in range(10):
218             name="Name"+str(i)
219             self.studentBoxes[name]=studentBox(self.boxesFrame,self,name,
    ↪ "F",["G"])
220             self.studentBoxes[name].pack(fill="x",padx=10,pady=5)
221
222     #Method override as a work-around for a scrolling bug
223     def update(self):
224         self.master.state('normal')
225         tk.Frame.update(self)
226         self.master.state('zoomed')
227
228     def check_for_selection(self):
229         for box in self.studentBoxes:
230             if self.studentBoxes[box].check_selection():
231                 return True
232         return False
233
234     def import_students(self):
235         #Show an "Open" dialog box and return an open file object in read
    ↪  mode
236         file = filedialog.askopenfile(mode='r',filetypes=[("Supported
    ↪ file types",".txt .csv")]) #Add excel
```

```
237         #Validation to ensure the user selected a file
238         if file == None:
239             return
240 
241         #<<<---------------------------------------------
242 
243         file.close()
244         return
245 
246     def new_student(self):
247         popupWin=entryPopupWin(self,"New student")
248         name=tk.StringVar()
249         gender=tk.StringVar()
250         skills=tk.StringVar()
251 
252         popupWin.add_string_entry("Enter name:", name)
253 
254         popupWin.add_string_entry("Enter gender:",gender)
255 
256         popupWin.add_string_entry("Enter skills, each one separated by a
    ↪ comma:",skills)
257 
258         popupWin.add_save_cancel(popupWin,name,gender,skills)
259 
260     def save_entry(self,window,*variables):
261         name=variables[0].get()
262         gender=variables[1].get()
263         skills=variables[2].get()
264 
265         #Validation to ensure all required entries are filled
266         if name != "" and gender != "" and skills != "":
267 
268             #Separate skills into a list and remove whitespace
269             skills=skills.split(",")
270             for i in range(len(skills)):
271                 skills[i]=skills[i].strip()
272 
273             #Display new student on the GUI
274             self.studentBoxes[name]=studentBox(self.boxesFrame,self,name,
    ↪ gender,skills)
275             self.studentBoxes[name].pack(fill="x",padx=10,pady=5)
276 
277             #Destroy the popup window
278             window.destroy()
279 
280             #Scroll bar bug work-around
281             self.update()
282 
283     def editBox(self,boxName):
284         return
```

```
285
286     def deleteBox(self,boxName):
287         self.studentBoxes[boxName].pack_forget()
288         del self.studentBoxes[boxName]
289         self.update()
290
291
292 class LoadingPage(tk.Frame):
293     def __init__(self,parent):
294         tk.Frame.__init__(self, parent)
295
296         self.label = ttk.Label(self, text="Grouping in progress...")
297         self.label.pack()
298
299         self.cancelButton=ttk.Button(self,text="Cancel",command=lambda:
    ↪ parent.show_frame(HomePage))
300         self.cancelButton.pack()
301
302 class ResultsPage(tk.Frame):
303     def __init__(self,parent):
304         tk.Frame.__init__(self, parent)
305
306         self.header=Header(self,"Results",HEADER_COLOUR)
307         self.header.pack(fill="x")
308
309         self.homeButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
310         self.homeButton.pack(side="right")
311
312 class RegisterPage(tk.Frame):
313     def __init__(self,parent):
314         tk.Frame.__init__(self, parent)
315
316         self.header=Header(self,"Register",HEADER_COLOUR)
317         self.header.pack(fill="x")
318
319         self.homeButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
320         self.homeButton.pack(side="right")
321
322 #=====WIDGETS=====
323
324 class Header(tk.Frame):
325     def __init__(self,parent,text,colour):
326         tk.Frame.__init__(self,parent)
327         self.config(bg=colour)
328         self.label = tk.Label(self, text=text, bg=colour,font=LARGE_FONT)
    ↪  #change to ttk
329         self.label.pack(side="left")
330
```

```
331 class ScrollableFrame(tk.Frame):
332     def __init__(self,parent):
333         self.canvas=tk.Canvas(parent,background="white")
334         tk.Frame.__init__(self,self.canvas)
335 
336         self.scrollbar=tk.Scrollbar(self.canvas,orient="vertical",command
    ↪ =self.canvas.yview)
337 
338         self.canvas.configure(yscrollcommand=self.scrollbar.set)
339 
340         self.scrollbar.pack(side="right",fill="y")
341         self.scrollbar.update()
342 
343         self.canvas.create_window((0,0),window=self,anchor='nw',width=
    ↪ self.winfo_screenwidth()-self.scrollbar.winfo_width())
344         self.canvas.bind("<Configure>",self.scroll)
345         #self.canvas.bind_all("<MouseWheel>", self._on_mousewheel)
346     #def _on_mousewheel(self, event):
347         #self.canvas.yview_scroll(int(-1*(event.delta/120)), "units")
348     def scroll(self,event):
349         self.canvas.configure(scrollregion=self.canvas.bbox("all"),width=
    ↪ self.winfo_width(),height=self.winfo_height())
350 
351 #Has checkbox, name, edit button, delete button.
352 class selectionBox(tk.Frame):
353     def __init__(self,parent,page,name):
354         tk.Frame.__init__(self,parent,relief="solid",bd=2)
355 
356         self.name=name
357         self.page=page
358 
359         self.selected=tk.BooleanVar(value=False)
360         self.checkbox=ttk.Checkbutton(self,variable=self.selected,name=
    ↪ name.lower(),onvalue=True,offvalue=False)
361         self.checkbox.pack()
362 
363         self.nameLabel=ttk.Label(self,text=name)
364         self.nameLabel.pack()
365 
366         self.editButton=ttk.Button(self,text="Edit",command=self.edit)
367         self.editButton.pack()
368 
369         self.deleteButton=ttk.Button(self,text="Delete",command=self.
    ↪ delete)
370         self.deleteButton.pack()
371 
372     def check_selection(self):
373         return self.selected.get()
374 
375     def edit(self):
```

```
376         self.page.editBox(self.name)
377 
378     def delete(self):
379         delChoice=messagebox.askyesno(title="Delete entry", message=f"Are
    ↪  you sure you want to delete {self.name}?")
380         if delChoice:
381             self.selected.set(False)
382             self.page.deleteBox(self.name)
383 
384 
385 class projectBox(selectionBox):
386     def __init__(self,parent,page,name,maxSize,minSize,optional,skills):
387         selectionBox.__init__(self,parent,page,name)
388 
389         self.maxSize=ttk.Label(self,text=f"Max group size: {maxSize}")
390         self.maxSize.pack()
391 
392         self.minSize=ttk.Label(self,text=f"Min group size: {minSize}")
393         self.minSize.pack()
394 
395         self.optional=ttk.Label(self,text=f"Optional: {optional}")
396         self.optional.pack()
397 
398         self.skills=ttk.Label(self,text=f"Skills: {", ".join(skills)}")
399         self.skills.pack()
400 
401 class studentBox(selectionBox):
402     def __init__(self, parent, page, name, gender, skills):
403         selectionBox.__init__(self,parent,page,name)
404 
405         self.gender = ttk.Label(self,text=f"Gender: {gender}")
406         self.gender.pack()
407 
408         self.skills = ttk.Label(self, text= f"Skills: {", ".join(skills)}
    ↪ ")
409         self.skills.pack()
410 
411 
412 class entryPopupWin(tk.Toplevel):
413     def __init__(self,parent,title):
414         tk.Toplevel.__init__(self,parent)
415         self.title(title)
416 
417     def add_string_entry(self,text,variable):
418         ttk.Label(self, text=text).pack()
419         ttk.Entry(self, textvariable=variable).pack()
420 
421     def add_int_entry(self,text,variable):
422 
```

```
423         #Prevents user from typing non numbers into number-only entry
    ↪ boxes
424         def intCallback(P):
425             return str.isdigit(P) or P == ""
426 
427         intValidate = (self.register(intCallback), '%P')
428 
429         ttk.Label(self, text=text).pack()
430         ttk.Entry(self, textvariable=variable,validate="all",
    ↪ validatecommand=intValidate).pack()
431 
432     def add_checkbox(self,text,variable):
433         ttk.Label(self, text=text).pack()
434         ttk.Checkbutton(self, variable=variable, onvalue=True, offvalue=
    ↪ False).pack()
435 
436     def add_save_cancel(self,window,*variables):
437         ttk.Button(self, text="Save", command=lambda:self.master.
    ↪ save_entry(window,*variables)).pack()
438         ttk.Button(self, text="Cancel", command=self.destroy).pack()
439 
440 
441 
442 #=====MAIN=====
443 
444 app=Window()
445 app.mainloop()
```

3.2 algoClasses.py

```
1  class Project:
2      def __init__(self,name,skills,maxSize,minSize,optional,group=None):
3          self.name=name
4          self.skills=skills
5          self.maxSize=maxSize
6          self.minSize=minSize
7          self.optional=optional
8  
9          self.group=group
10         self.fitness=self.calcFitness()
11 
12     def calcFitness():
13         return
14 
15 class Student:
16     def __init__(self,name,skills,gender):
17         self.name=name
```

```
        self.skills=skills
        self.gender=gender

class Individual:
    def __init__(self,projects):
        self.projects=projects
        self.fitness=self.calcFitness()
    def calcFitness(self):
        return
```

3.3 geneticAlgo.py

```
from algoClasses import Project,Student,Individual
import sqlite3 as sql

def get_students(database,IDs):
    students=[]

    with sql.connect(database) as conn:
            cur = conn.cursor()


            for ID in IDs:
                #Get each students' skills
                cur.execute("SELECT Skill FROM StudentSkills WHERE
    ↪ StudentID = ?",(ID))
                skills=cur.fetchall()

                #Get each students' name and gender
                cur.execute("SELECT Name, Gender from Students WHERE
    ↪ StudentID = ?",(ID))
                entry=cur.fetchall()

                #Create a Student object for each student with their name
    ↪ , skills, and gender
                students.append(Student(entry[0],skills,entry[1]))

    return students


def get_projects(database,IDs):
    projects=[]

    with sql.connect(database) as conn:
            cur = conn.cursor()

            for ID in IDs:
```

```
33                  #Get each projects' skill requirements
34                  cur.execute("SELECT Skill FROM ProjectSkills WHERE
    ↪ ProjectID = ?",(ID))
35                  skills=cur.fetchall()
36
37                  #Get each projects' data other than skills
38                  cur.execute("SELECT Name, MaxSize, MinSize, Optional from
    ↪  Projects WHERE ProjectID = ?",(ID))
39                  entry=cur.fetchall()
40
41                  #Create a Project object for each project with its name,
    ↪ skills, maxSize, minSize, and whether or not it is optional
42                  projects.append(Project(entry[0],skills,entry[1],entry[2
    ↪ ],entry[3]))
43
44      return projects
45
46
47
48  def run(database, studentIDs, projectIDs, target_fitness, max_iterations,
    ↪  population_size):
49      students=get_students(database,studentIDs)
50      projects=get_projects(database,projectIDs)
51
52      #Ensure there aren't too many or too few students to meet
    ↪ requirements
```

4 Testing

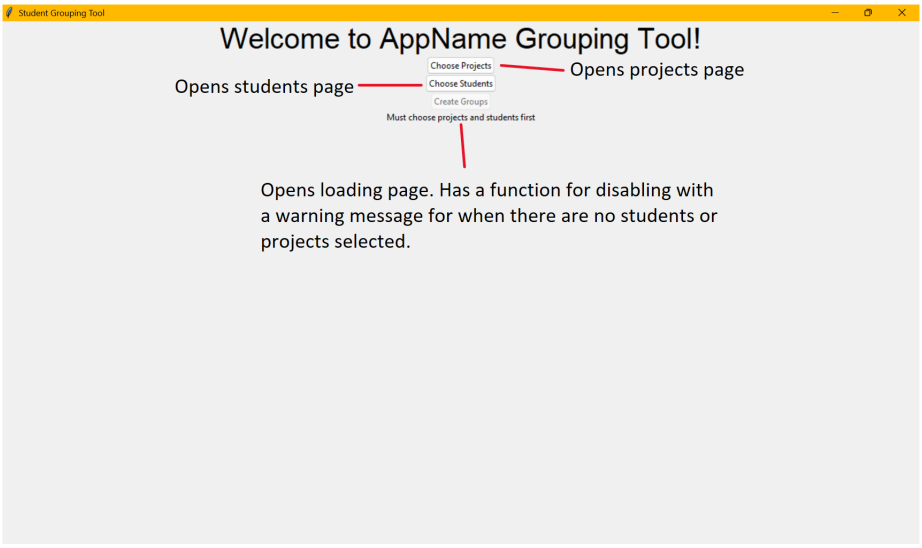


Figure 19: The home page

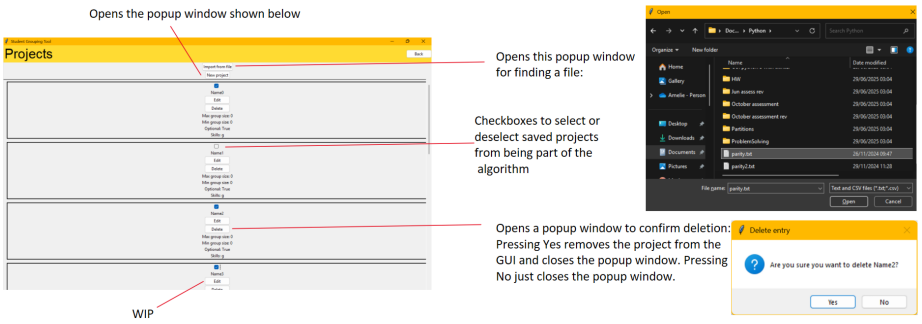


Figure 20: The project page

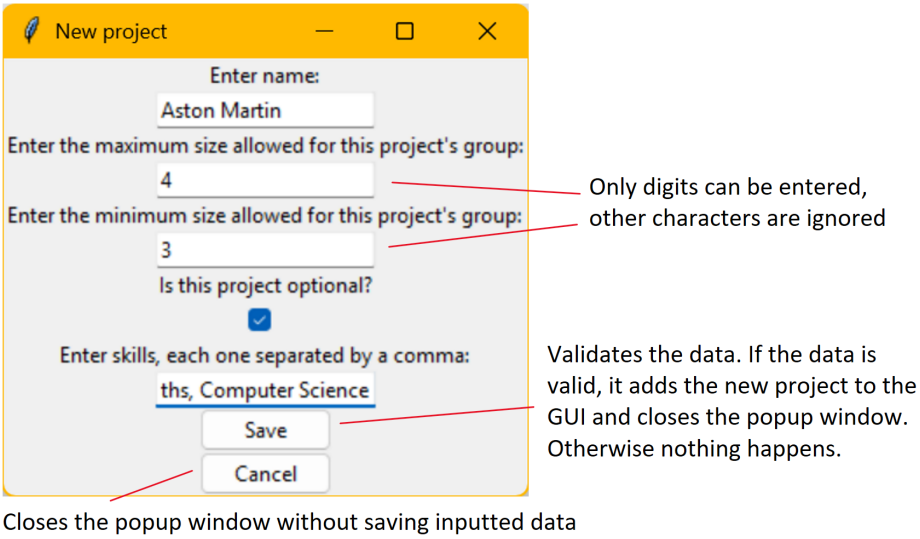


Figure 21: The popup window for entering data about a new project

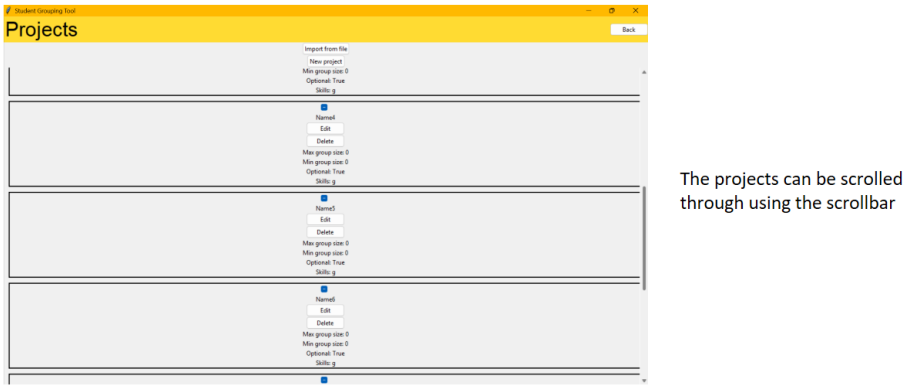


Figure 22: The project page, scrolled down

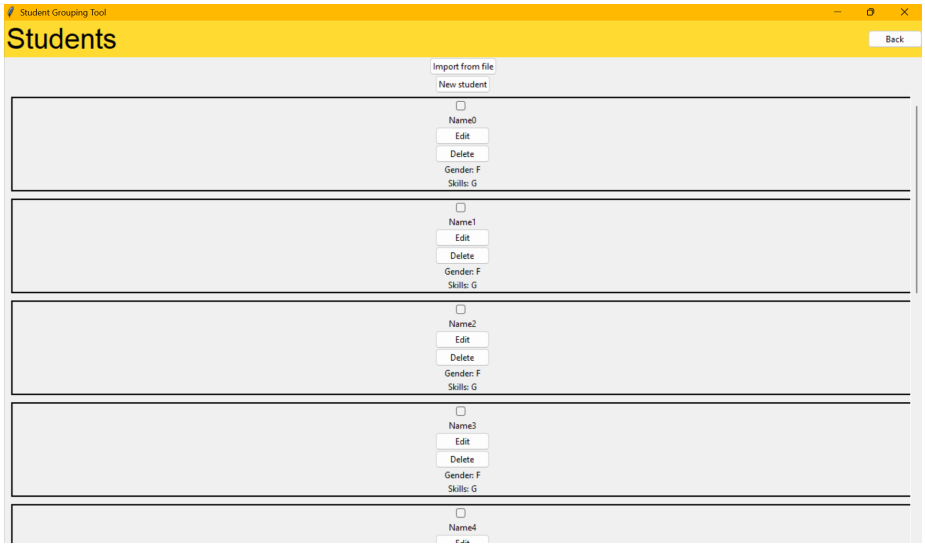


Figure 23: The student page. Functions identically to the projects page

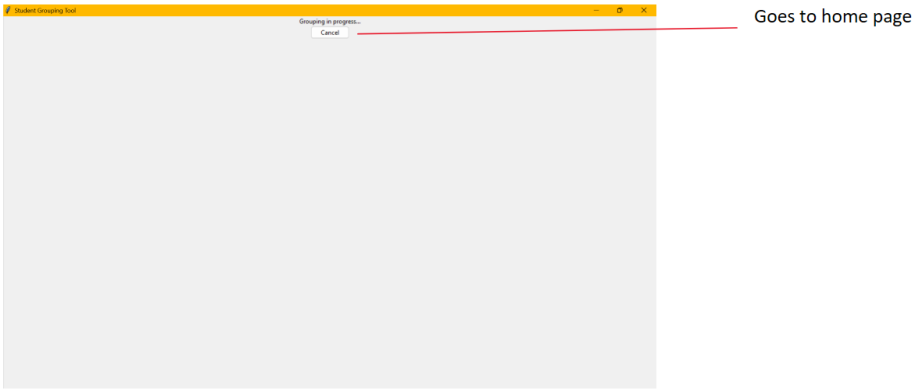


Figure 24: The loading page

5 Evaluation

So far, I have created a basic GUI with minimal formatting, and some basic functionality such as page navigation, scrollable frames, project management and student management.

I have also created the classes required by the genetic algorithm and made a start coding up my pseudocode for the genetic algorithm.

The fitness algorithm is harder to design than I thought it would be. I got a bit carried away working on the GUI and didn't spend as much time as I needed on the fitness algorithm so I haven't got one yet. I also didn't add much to the design section because of this, so I will add more diagrams to the design section before the next submission.